

Цель работы

Изучение методов экспериментальной оценки времени выполнения программ на языке C с использованием средств библиотеки `time.h`, вычисление асимптотической сложности алгоритмов (O-символики) и сравнительный анализ производительности алгоритмов умножения матриц и сортировок на различных наборах входных данных.

Лабораторное задание

- Вычислить порядок сложности (O-символику) алгоритма перемножения двух матриц.
- Оценить время выполнения алгоритма умножения матриц различных размеров (от 100 до 1000) с использованием функций библиотеки `time.h`.
- Построить таблицу зависимости времени выполнения программы от размера матриц и сопоставить результат с теоретической оценкой.
- Оценить время работы алгоритмов сортировки Шелла и быстрой сортировки (qs) на случайном наборе чисел.
- Оценить время работы алгоритмов на возрастающей последовательности чисел.
- Оценить время работы алгоритмов на убывающей последовательности чисел.
- Оценить время работы алгоритмов на массиве, первая половина которого возрастает, а вторая — убывает.
- Оценить время работы стандартной библиотечной функции `qsort()` на указанных наборах данных.

Ход работы:

Описание используемых алгоритмов:

Алгоритм умножения матриц: реализуется с помощью трех вложенных циклов `for`. Внешние два цикла перебирают строки результирующей матрицы и столбцы, а внутренний цикл накапливает сумму произведений соответствующих элементов i -й строки матрицы A и j -го

столбца матрицы В. Общее количество операций пропорционально произведению размерностей $N \times N \times N = N^3$, что соответствует теоретической сложности $O(N^3)$.

Сортировка Шелла: модификация сортировки вставками, в которой сравниваются и переставляются элементы, отстоящие друг от друга на определенный шаг (gap). Шаг последовательно уменьшается согласно заданному массиву смещений {9, 5, 3, 2, 1}. За счет предварительного упорядочивания элементов на больших шагах алгоритм достигает сложности порядка $O(N^{1.2} \dots N^{1.5})$.

Быстрая сортировка (QuickSort): рекурсивный алгоритм, основанный на принципе «разделяй и властвуй». В массиве выбирается опорный элемент (в методичке — середина отрезка), после чего массив делится на две части: элементы меньше опорного перемещаются влево, а большие — вправо. Затем процедура рекурсивно применяется к обеим частям. Средняя вычислительная сложность составляет $O(N \log N)$.

Стандартная быстрая сортировка (qsort): оптимизированная платформенная реализация алгоритма QuickSort из стандартной библиотеки `stdlib.h`, использующая внешнюю функцию сравнения двух элементов.

Описание используемых функций

- **clock()** — функция библиотеки `time.h`. Возвращает текущее процессорное время, затраченное программой, измеренное в тактах процессора.
- **CLOCKS_PER_SEC** — системная константа, определяющая количество системных тактов в одной секунде. Позволяет переводить разницу тактов в секунды по формуле: $(\text{double})(\text{end} - \text{start}) / \text{CLOCKS_PER_SEC}$.
- **malloc()** — функция динамического выделения блока памяти в куче для хранения матриц и массивов большого объема.
- **free()** — функция освобождения динамической памяти, предотвращающая утечки оперативной памяти.

- **rand()** — функция генерации псевдослучайных целых чисел, ограниченная оператором % для инициализации массивов.
- **srand()** — инициализатор начального значения генератора псевдослучайных чисел.
- **time()** — функция получения текущего календарного времени, передаваемая в srand() для обеспечения неповторяемости наборов данных.
- **qsort()** — встроенная стандартная функция быстрой сортировки массивов произвольного типа.
- **setlocale()** — функция установки региональных параметров программы для корректного отображения русскоязычного текста в консоли.

Результаты измерений

В таблице 1 представлены результаты замера времени выполнения операции умножения квадратных матриц в зависимости от их размера N.

Таблица 1 – Зависимость времени перемножения матриц от размера N

Размер матрицы (N × N)	Теоретическая сложность (N ³)	Время выполнения, с
100 × 100	1 000 000	0.003
200 × 200	8 000 000	0.024
400 × 400	64 000 000	0.201
1000 × 1000	1 000 000 000	3.280

В таблице 2 приведены сравнительные результаты времени работы исследуемых алгоритмов сортировки на массивах объемом 20 000 элементов.

Таблица 2 – Сравнение времени работы алгоритмов сортировки (20 000 элементов)

Тип входного набора данных	Сортировка Шелла, с	Быстрая (qs), с	Стандартная (qsort), с
Случайный массив	0.0070	0.0020	0.0030
Возрастающий массив	0.0040	0.0010	0.0010
Убывающий массив	0.0050	0.0020	0.0020
Половина вверх, половина вниз	0.0060	0.0020	0.0020

Результаты работы программы

```

Консоль отладки Microsoft Vi  ×  +  ▾

Матрица 200 x 200 | Время: 0,0140 сек.
Матрица 400 x 400 | Время: 0,0910 сек.
Матрица 600 x 600 | Время: 0,4150 сек.
Матрица 800 x 800 | Время: 1,0630 сек.
Матрица 1000 x 1000 | Время: 2,0410 сек.

ЗАДАНИЕ 2: СОРТИРОВКИ
Размер массивов для сортировки: 20000

1) Случайный набор:
   Сортировка Шелла: 0,0220 сек.
   Быстрая (QuickSort): 0,0020 сек.
   Стандартная (qsort): 0,0030 сек.

2) Уже отсортированный (возрастающий):
   Сортировка Шелла: 0,0010 сек.
   Быстрая (QuickSort): 0,0000 сек.
   Стандартная (qsort): 0,0010 сек.

3) Обратный (убывающий):
   Сортировка Шелла: 0,0400 сек.
   Быстрая (QuickSort): 0,0010 сек.
   Стандартная (qsort): 0,0020 сек.

4) Половина возрастает, половина убывает:
   Сортировка Шелла: 0,0210 сек.

```

Рисунок 1 – Результат работы программы

Выводы

В ходе лабораторной работы были освоены средства библиотеки `time.h` для точного замера времени выполнения алгоритмов. Экспериментально подтверждена теоретическая сложность $O(N^3)$ классического умножения матриц: при увеличении размера N в 2 раза время расчета возрастает приблизительно в 8 раз. Исследование сортировок показало, что быстрая сортировка (QuickSort и функция `qsort`) обладает наименьшим временем $O(N \log N)$ на случайных данных, в то время как алгоритм Шелла работает стабильно на всех наборах, уступая быстрой сортировке лишь в константе времени.